

ReactJS Hands-on - Trainers App with React Router

Navigation menu, routes, and URL parameter passing using react-router-dom

1. Objective

Build an SPA called **TrainersApp** that maintains a list of Cognizant Academy trainers and their expertise, using **React Router** for navigation between a Home page, a Trainers List page, and a Trainer Details page (with the trainer id passed via the URL).

2. Prerequisites

- Node.js
- NPM
- Visual Studio Code

3. Create the React App

● ● ● Command Prompt

```
C:\Users\student> npx create-react-app TrainersApp
Creating a new React app in C:\Users\student\TrainersApp.
Installing react, react-dom, and react-scripts...

Success! Created TrainersApp at C:\Users\student\TrainersApp
Happy hacking!

C:\Users\student> cd TrainersApp
C:\Users\student\TrainersApp> code .
```

4. Trainer Class - trainer.js

Added trainer.js in the **src** folder defining the Trainer class with the required properties.

src/trainer.js

```
class Trainer {
  constructor(trainerId, name, email, phone, technology, skills) {
    this.trainerId = trainerId;
    this.name = name;
    this.email = email;
    this.phone = phone;
    this.technology = technology;
    this.skills = skills;
  }
}

export default Trainer;
```

5. Mock Data - TrainersMock.js

src/TrainersMock.js

```
import Trainer from './trainer';

const trainersMock = [
  new Trainer(101, "Anil Kumar", "anil.kumar@cognizant.com", "9876543210",
    "React", ["JavaScript", "React", "Redux"]),
  new Trainer(102, "Priya Sharma", "priya.sharma@cognizant.com", "9876543211",
    ".Net", ["C#", "ASP.Net Core", "SQL Server"]),
  new Trainer(103, "Rahul Verma", "rahul.verma@cognizant.com", "9876543212",
    "Java", ["Java", "Spring Boot", "Microservices"]),
  new Trainer(104, "Sneha Rao", "sneha.rao@cognizant.com", "9876543213",
    "React", ["React", "Node.js", "MongoDB"])
];

export default trainersMock;
```

6. Install React Router

● ● ● Command Prompt

```
C:\Users\student\TrainersApp> npm install react-router-dom
added 1 package, and audited 1391 packages in 6s
found 0 vulnerabilities
```

7. TrainersList Component

Accepts the trainer list as a prop and renders each trainer's name as a clickable link.

src/Trainerlist.js

```
import React from 'react';
import { Link } from 'react-router-dom';

function TrainersList({ trainers }) {
  return (
    <div>
      <h2>Trainers List</h2>
      <ul>
        {trainers.map((trainer) => (
          <li key={trainer.trainerId}>
            <Link to={`/${trainer.trainerId}`}>{trainer.name}</Link>
          </li>
        ))}
      </ul>
    </div>
  );
}

export default TrainersList;
```

8. Home Component

src/Home.js

```
import React from 'react';

function Home() {
  return (
    <div>
      <h2>Welcome to Cognizant Academy Trainers App</h2>
    </div>
  );
}

export default Home;
```

9. App Component - Routing

Configured BrowserRouter, Routes, Route and Link so that / shows Home and /trainers shows the trainers list.

src/App.js

```
import React from 'react';
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
import Home from './Home';
import TrainersList from './Trainerlist';
import TrainerDetail from './TrainerDetails';
import trainersMock from './TrainersMock';

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link> | <Link to="/trainers">Trainers</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/trainers" element={<TrainersList trainers={trainersMock} />} />
        <Route path="/trainers/:id" element={<TrainerDetail />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

10. TrainerDetail Component - URL Parameter

Reads the id from the URL using the useParams hook and looks up the matching trainer from the mock data.

src/TrainerDetails.js

```
import React from 'react';
import { useParams } from 'react-router-dom';
import trainersMock from './TrainersMock';

function TrainerDetail() {
  const { id } = useParams();
  const trainer = trainersMock.find((t) => t.trainerId === parseInt(id));
```

```

    if (!trainer) {
      return <p>Trainer not found</p>;
    }

    return (
      <div>
        <h2>Trainer Details</h2>
        <p><b>ID:</b> {trainer.trainerId}</p>
        <p><b>Name:</b> {trainer.name}</p>
        <p><b>Email:</b> {trainer.email}</p>
        <p><b>Phone:</b> {trainer.phone}</p>
        <p><b>Technology:</b> {trainer.technology}</p>
        <p><b>Skills:</b> {trainer.skills.join(', ')}</p>
      </div>
    );
  }
}

export default TrainerDetail;

```

Update TrainersList to Link to Details

```

<Link to={`/trainers/${trainer.trainerId}`}>{trainer.name}</Link>

```

(already included in the TrainersList code above – each trainer name links to /trainers/<id>, which is handled by the /trainers/:id route.)

11. Build and Run the Application

Command Prompt

```

C:\Users\student\TrainersApp> npm start
Compiled successfully!

You can now view TrainersApp in the browser.
Local: http://localhost:3000

```

Home Page - "/"

localhost:3000/

[Home](#) | [Trainers](#)

Welcome to Cognizant Academy Trainers App

Trainers List Page - "/trainers"

localhost:3000/trainers

[Home](#) | [Trainers](#)

Trainers List

- [Anil Kumar](#)
- [Priya Sharma](#)
- [Rahul Verma](#)
- [Sneha Rao](#)

Trainer Details Page - "/trainers/101"

localhost:3000/trainers/101

[Home](#) | [Trainers](#)

Trainer Details

ID: 101

Name: Anil Kumar

Email: anil.kumar@cognizant.com

Phone: 9876543210

Technology: React

Skills: JavaScript, React, Redux

12. Observation

BrowserRouter, Routes and Route set up client-side navigation without full page reloads. The Link component swaps the URL and renders the matching route's component, and useParams lets TrainerDetail read the id segment straight from the URL so the same component can display any trainer without a

separate route per trainer.